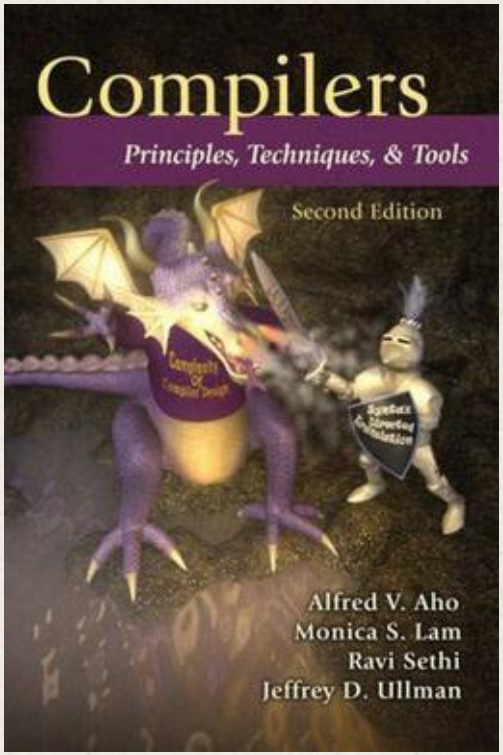
CS407 Lecture 1

Introduction

TextBook

- **Compilers: Principles, Techniques, and Tools, Aho, Sethi, Ullman (aka red dragon book) second edition.**

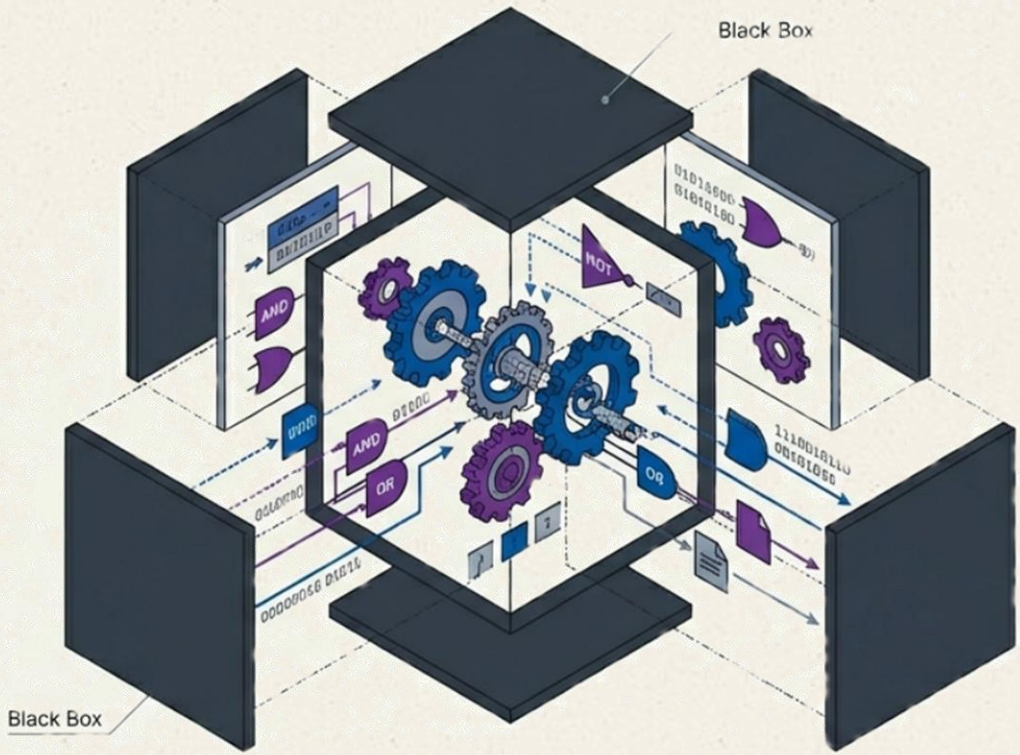


Course Grading

Component	Grade	
Lab Work	30%	20% Exams 10% Project
Midterm	10%	
Final Exam	60%	
Bonus	2%	Class participation

Course Goal

1. Open the lid to see **inside**
2. Understand **functionality** (what they do)
3. Understand **mechanics** (how they work)
4. Understand **implementation** (how to build them)



How are Languages Implemented?

- Two major strategies:
  - Interpreters run your program
  - Compilers translate your program

Compilers

- **A compiler** is a program that can read a program in one language - the source language - and translate it into an equivalent program in another language - the target language.

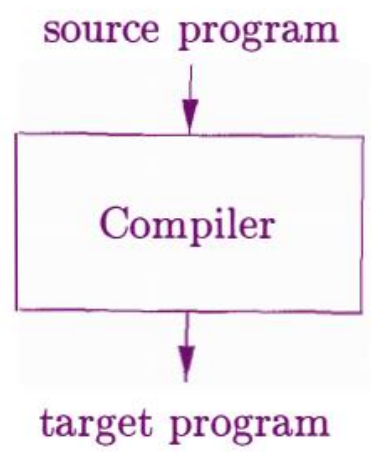


Figure 1.1: A compiler

Compilers

- report any errors in the source program that it detects during the translation process.
- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs

Figure 1.2: Running the target program

Interpreters

- An interpreter is another common kind of language processor.
- Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.

Figure 1.3: An interpreter

Compilers VS Interpreters

- The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs .
- An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

Compilers VS Interpreters

- The first interpreted high-level language was Lisp. Python, Ruby, Perl and PHP are other examples of programming languages that use interpreters.

- C, C++, C#, are examples of programming languages that are compiler-based.

Example: A hybrid compiler

- Java language processors combine compilation and interpretation.
- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine.

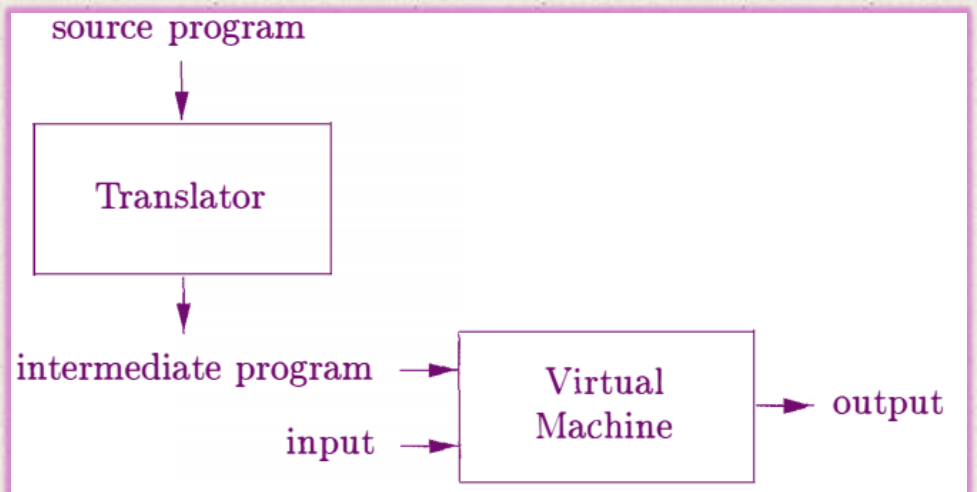


Figure 1.4: A hybrid compiler

Example: A hybrid compiler

- In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input.

A language-processing system

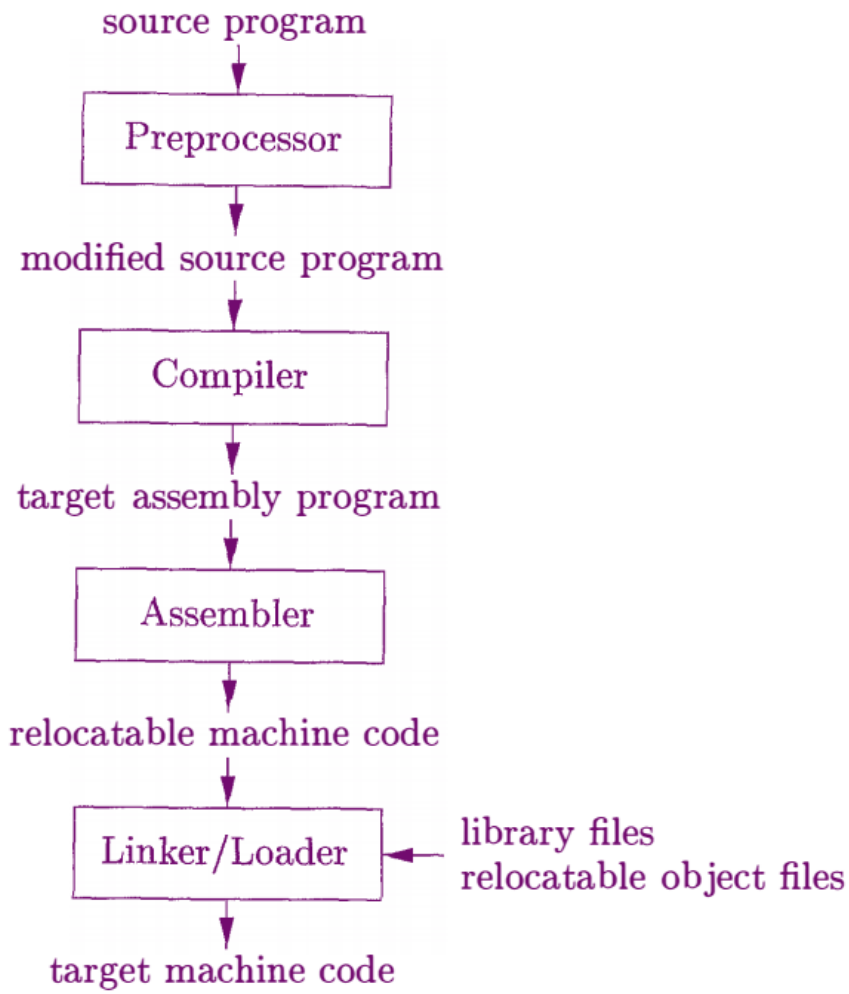


Figure 1.5: A language-processing system

Preprocessor

- A source program may be divided into modules stored in separate files.
- The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor.
- The preprocessor may also expand shorthands, called macros, into source language statements.

Assembler

- The compiler may produce an assembly-language program as its output, because assembly language is easier to produce as output and is easier to debug.
- The assembly language is then processed by a program called an assembler that produces relocatable machine code as its output.

Linker and Loader

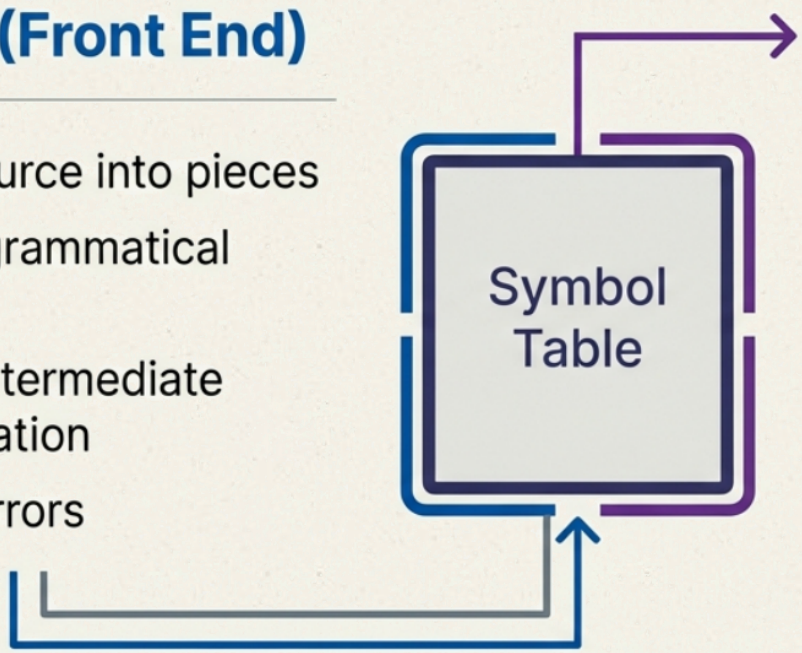
- Large programs are often compiled in pieces, so the relocatable machine code may have to be linked together with other relocatable object files and library files into the code that actually runs on the machine.
- The linker resolves external memory addresses, where the code in one file may refer to a location in another file.
- The loader then puts together all of the executable object files into memory for execution.

The Structure of a Compiler

Two parts: analysis (front end) and synthesis (back end).

Analysis (Front End)

- Breaks source into pieces
- Imposes grammatical structure
- Creates intermediate representation
- Detects errors



Synthesis (Back End)

- Constructs target program
- Generates machine code

Phases of a compiler

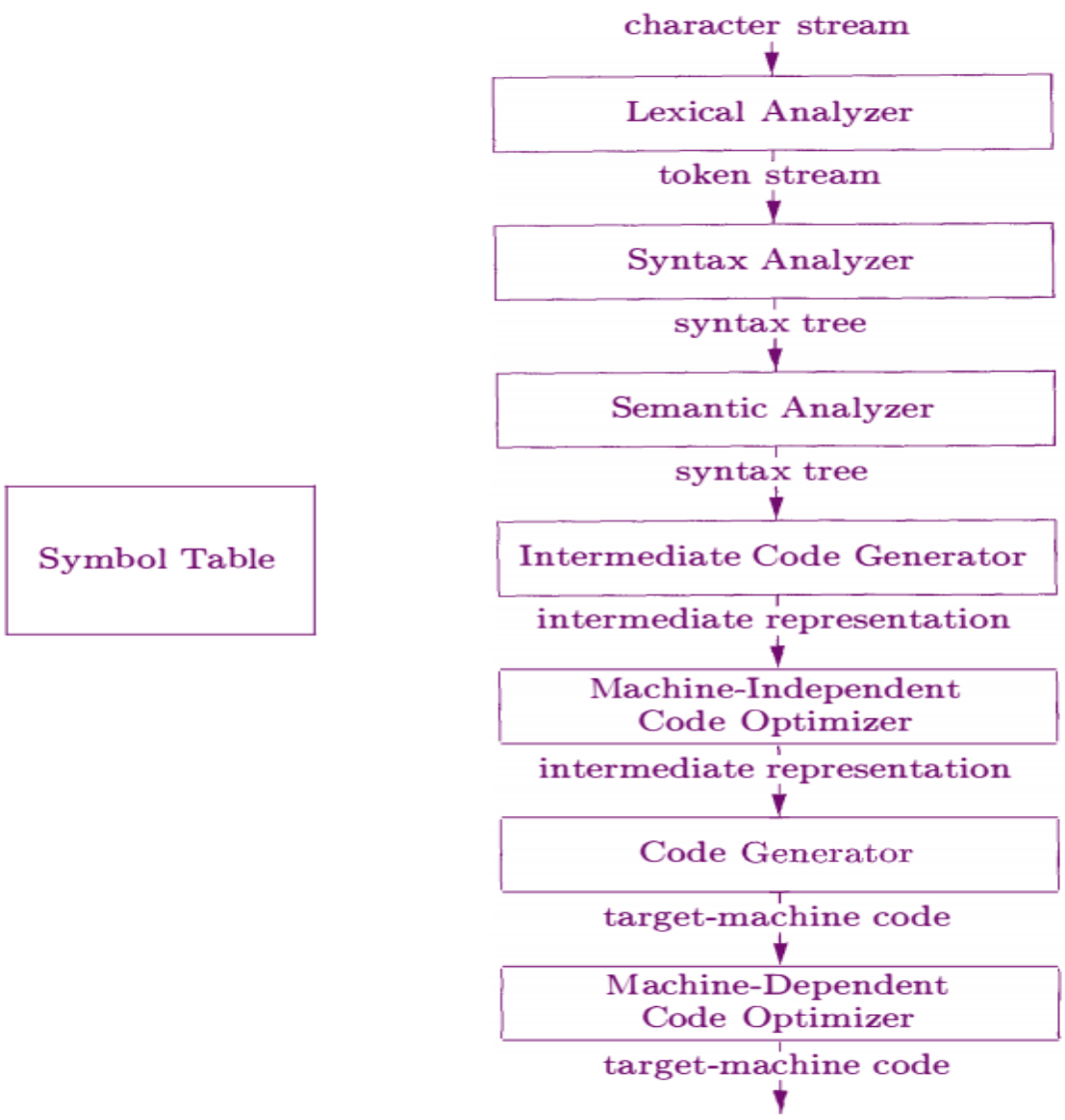


Figure 1.6: Phases of a compiler

Lexical Analysis (Scanning)

- The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes.
- For each lexeme, the lexical analyzer produces as output a token of the form

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

that it passes on to the subsequent phase, syntax analysis.

Lexical Analysis: Tokens

- The first component token-name is an abstract symbol that is used during syntax analysis.
- The second component attribute-value points to an entry in the symbol table for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

Lexical Analysis: Tokens

- For example, suppose a source program contains the assignment statement:

```
position = initial + rate * 60                    (1.1)
```

- The characters in this assignment could be grouped into lexemes and mapped into the following tokens passed on to the syntax analyzer:

$$\langle \mathbf{id}, 1\rangle \ \langle = \rangle \ \langle \mathbf{id}, 2\rangle \ \langle + \rangle \ \langle \mathbf{id}, 3\rangle \ \langle * \rangle \ \langle 60 \rangle \qquad (1.2)$$

- Blanks separating the lexemes would be discarded by the lexical analyzer.

Lexical Analysis: Tokens

$$\langle \mathbf{id}, 1\rangle \ \langle = \rangle \ \langle \mathbf{id}, 2\rangle \ \langle + \rangle \ \langle \mathbf{id}, 3\rangle \ \langle * \rangle \ \langle 60 \rangle \tag{1.2}$$

identifier

the symbol-table entry

Lexical Analysis

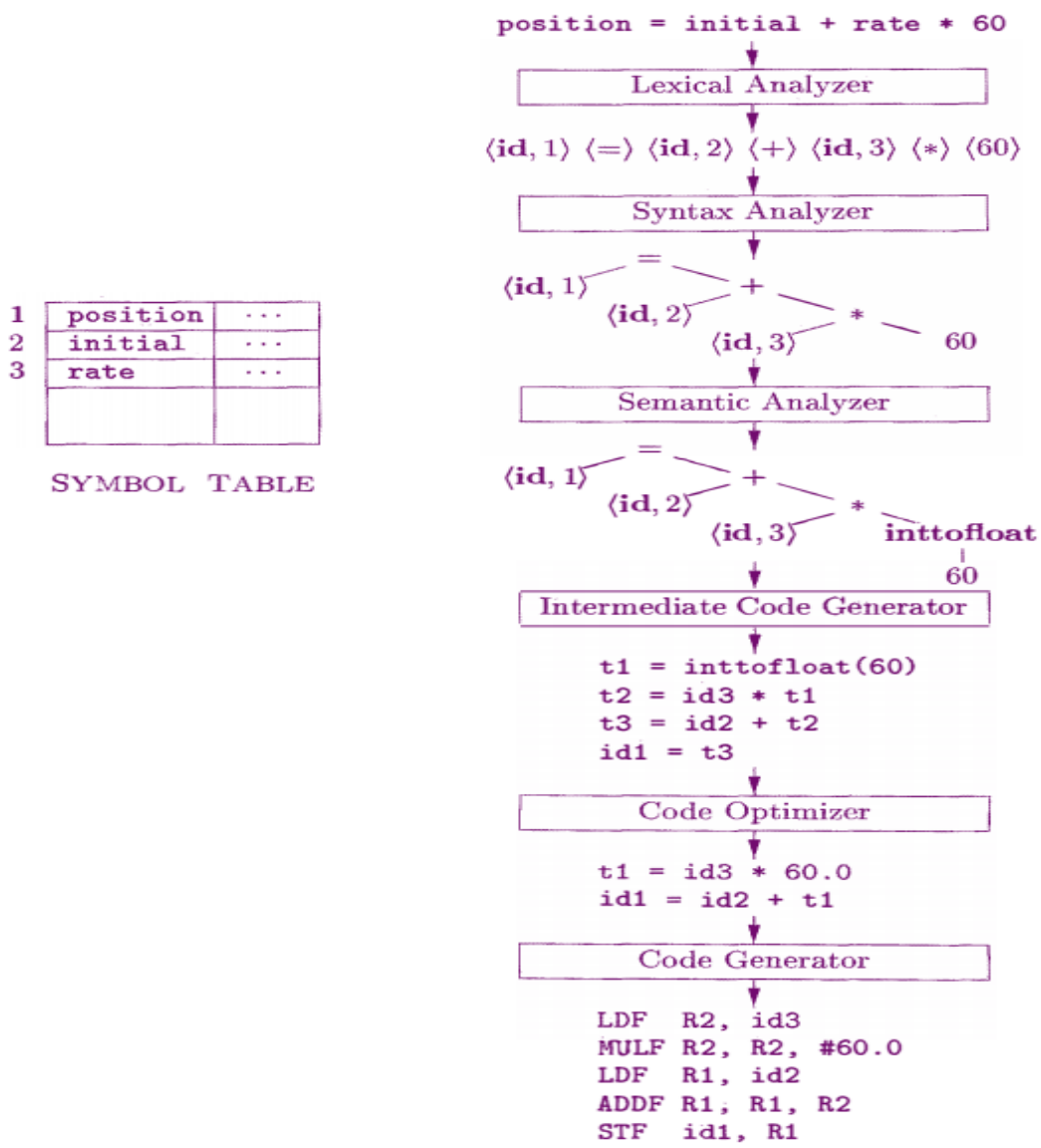


Figure 1.7: Translation of an assignment statement

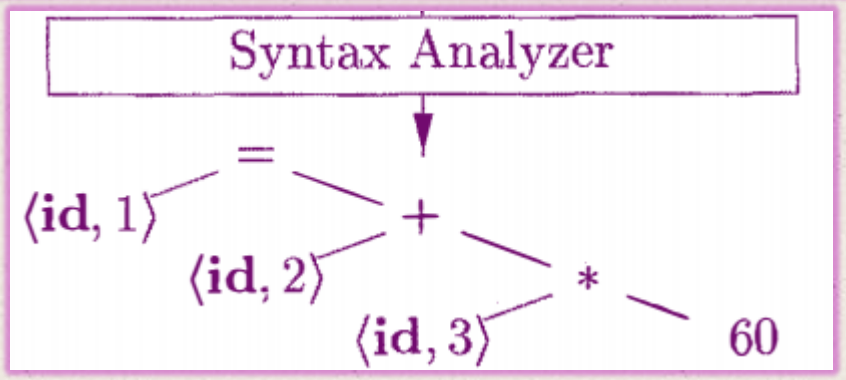
Syntax Analysis (Parsing)

- The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.
- A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.

Syntax Analysis (Parsing)

- A syntax tree for the token stream (1.2) is shown as the output of the syntactic analyzer in Fig. 1.7.
- This tree shows the order in which the operations in the assignment are to be performed.

```
position = initial + rate * 60                    (1.1)
```



Semantic Analysis

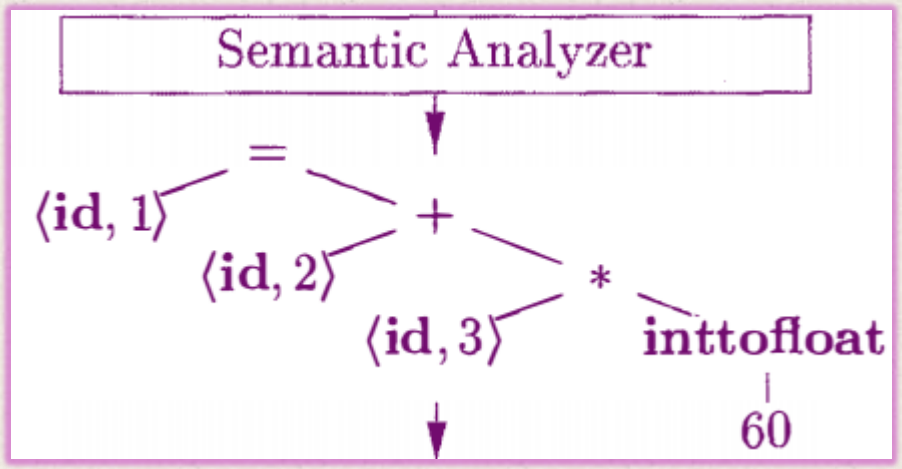
- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
- For example, many programming language definitions require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.

Semantic Analysis

- The language specification may permit some type conversions called coercions.
- For example, a binary arithmetic operator may be applied to either a pair of integers or to a pair of floating-point numbers.
- If the operator is applied to a floating-point number and an integer, the compiler may convert or coerce the integer into a floating-point number.
- Such a coercion appears in Fig. 1.7. Suppose that position, initial, and rate have been declared to be floating-point numbers, and that the lexeme 60 by itself forms an integer.
- The type checker in the semantic analyzer in Fig. 1.7 discovers that the operator * is applied to a floating-point number rate and an integer 60. In this case, the integer may be converted into a floating-point number.

Semantic Analysis

- In Fig. 1 .7, notice that the output of the semantic analyzer has an extra node for the operator **inttofloat**, which explicitly converts its integer argument into a floating-point number.



Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation.
- This intermediate representation should have two important properties:
  - it should be easy to produce,
  - it should be easy to translate into the target machine.

Intermediate Code Generation

- We consider an intermediate form called three-address code.
- It consists of a sequence of assembly-like instructions with three operands per instruction.
- Each operand can act like a register.

Intermediate Code Generation

- The output of the intermediate code generator in Fig. 1.7 consists of the three-address code sequence:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

(1.3)

- **In three-address instructions**:
  - Each assignment instruction has at most one operator on the right side (fix the order).
  - The compiler must generate a temporary name to hold the value computed by a three-address instruction.
  - Some "three-address instructions" like the first and last in (1.3) have fewer than three operands.

Code Optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result. Usually better means faster, but other objectives may be desired, such as shorter code, or target code that consumes less power.
- **In our example:**
- The optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time.
- so the inttofloat operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- Moreover, t3 is used only once to transmit its value to id1.

Code Optimization

- So the optimizer can transform (1.3) into the shorter sequence:

```
t1 = id3 * 60.0
id1 = id2 + t1
```
(1.4)

Code Generation

- The code generator takes as input an intermediate representation of the source program and maps it into the target language.
- If the target language is machine code, registers Or memory locations are selected for each of the variables used by the program.
- Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.

Code Generation

- For example, using registers R1 and R2, the intermediate code in (1.4) might get translated into the machine code

```
LDF  R2,  id3
MULF R2,  R2, #60.0
LDF  R1,  id2                    (1.5)
ADDF R1,  R1, R2
STF  id1, R1
```

Code Generation

```
LDF  R2,  id3
MULF R2,  R2, #60.0
LDF  R1,  id2                    (1.5)
ADDF R1,  R1, R2
STF  id1, R1
```

- The first operand of each instruction specifies a destination.
- The F in each instruction tells us that it deals with floating-point numbers.
- The code in (1.5) loads the contents of address id3 into register R2,
- then multiplies it with floating-point constant 60.0.
- The # signifies that 60.0 is to be treated as an immediate constant.
- The third instruction moves id2 into register Rl and the fourth adds to it the value previously computed in register R2.
- Finally, the value in register Rl is stored into the address of idl.

Symbol-Table Management

- An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name.
- **These attributes may provide information about**:
  - The storage allocated for a name, its type, its scope (where in the program its value may be used),
  - and in the case of procedure names, such things as the number and types of its arguments, the method of passing each argument (for example, by value or by reference), and the type returned.

Symbol-Table Management

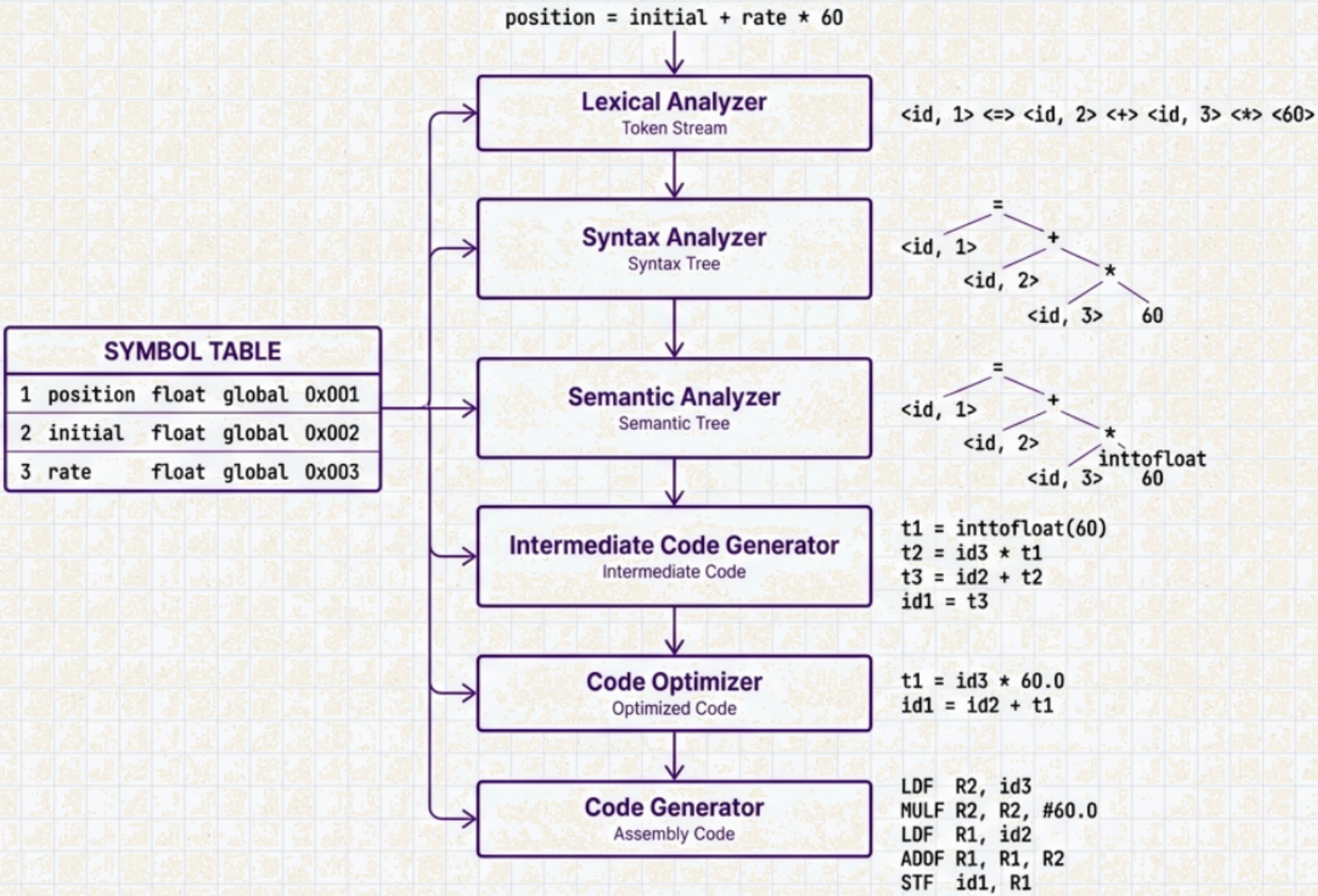
- The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name.
- The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.

Symbol-Table Management

Variable Name	Type	Scope	Memory Address
position	float	global	0x001
initial	float	global	0x002
rate	float	global	0x003

A persistent database recording every variable's identity. Essential for quick retrieval during Analysis and Synthesis.

Summary: The Complete Transformation



SYMBOL TABLE				
1	position	float	global	0x001
2	initial	float	global	0x002
3	rate	float	global	0x003